Research Report: FP16 persistent weights (model.half()) vs FP32 weights with FP16 autocast for neural network inference quality - specifically for image segmentation, matting, and alpha prediction models. Evidence of PSNR, SSIM, MSE quality degradation. Does model.half() cause any measurable quality loss compared to keeping FP32 weights with torch.autocast? Include real-world benchmarks, papers, and production deployment evidence.

Generated: 03/12/2026, 19:05:32
Sources: 65 verified

Executive Summary

Converting PyTorch vision model weights to persistent FP16 via `model.half()` causes measurable, and in some cases catastrophic, quality degradation compared to retaining FP32 weights and using `torch.autocast` for mixed-precision inference. A blanket `model.half()` conversion forces all operations, including numerically sensitive ones, into the limited FP16 range, leading to instability. Documented failures are common in `BatchNorm` and `LayerNorm` layers, `Softmax` operations, and the MLP/FFN blocks of vision transformers. In one documented case, a `model.half()` conversion on a segmentation model's encoder resulted in a "catastrophic failure" with a near-zero mean Intersection over Union (mIoU) score.

In contrast, `torch.autocast` is an intelligent context manager that automatically keeps these sensitive operations in full FP32 precision, preventing such failures and preserving model quality with what is often a negligible loss of less than 1%. While no public benchmarks directly compare the two methods using PSNR or SSIM metrics for segmentation or matting, the evidence from mIoU degradation, combined with official guidance from PyTorch and NVIDIA, is conclusive. The production-standard best practice is to use `torch.autocast` or to provide a full FP32 model to an inference engine like TensorRT and allow its optimizer to manage the mixed-precision conversion, a process that mirrors the `autocast` safety-first philosophy.

Key Findings

1. Finding: A blanket `model.half()` conversion can cause catastrophic quality loss in segmentation models, with one study reporting a near-zero mIoU score on an EfficientViT-SAM model, while `torch.autocast` is designed to maintain accuracy.

Confidence: MEDIUM
Sources: Multiple sources

2. Confidence: HIGH

Confidence: HIGH
Sources: Multiple sources

3. Sources: [29, 30]

Confidence: MEDIUM
Sources: Multiple sources

4. Finding: Normalization layers (`BatchNorm` , `LayerNorm`) and the `Softmax` function are the most frequently documented sources of numerical instability and model failure when an entire model is converted to FP16.

Confidence: MEDIUM
Sources: Multiple sources

5. Confidence: HIGH

Confidence: HIGH
Sources: Multiple sources

6. Sources: [3, 19, 21, 22]

Confidence: MEDIUM
Sources: Multiple sources

7. Finding: In vision transformer models like SegFormer, the MLP (Multi-Layer Perceptron) / FFN (Feed-Forward Network) blocks are the components most sensitive to FP16 quantization errors, contributing more to precision loss than the attention or normalization layers.

Confidence: MEDIUM
Sources: Multiple sources

8. Confidence: HIGH

Confidence: HIGH
Sources: Multiple sources

Detailed Analysis

The Fundamental Difference: Blanket vs. Selective Precision

The core of the quality difference lies in the implementation of the two methods. `model.half()` is a blanket operation that recursively traverses all modules in a model and casts all floating-point parameters and buffers to FP16 precision [11, 24]. This is a naive, brute-force approach that does not account for the numerical properties of different operations.

In contrast, `torch.autocast` is a context manager that enables automatic mixed precision [20]. It does not persistently change the model's weights. Instead, during the forward pass, it selectively casts inputs for specific operations to FP16 to gain performance while keeping others in FP32 to maintain stability and accuracy [27]. PyTorch maintains an internal list of "safe" operations (e.g., convolutions, matrix multiplications) to run in FP16 and a list of operations that should remain in FP32 (e.g., reduction functions, `Softmax`) [3, 22]. This intelligent, selective approach is the key to preserving model quality.

Identified Failure Points in Vision Models

Research and production experience have consistently identified several types of layers that are prone to failure under a full FP16 conversion.

* **Normalization Layers (`BatchNorm`, `LayerNorm`):** These layers are the most cited source of instability. They compute statistics (mean, variance) that often involve summing up many small numbers, a process highly susceptible to numerical underflow (values becoming zero) in FP16's limited range [19, 21]. While modern PyTorch implementations of `BatchNorm` compute these statistics in FP32 even for FP16 inputs to enhance stability, a blanket `model.half()` can override this safeguard [23]. Manually casting these layers back to FP32 after a `.half()` call is a common, documented workaround [2, 23].
* **Softmax and Attention Mechanisms:** The `Softmax` function, critical for the final prediction layer in segmentation models and for attention blocks in transformers, can fail in FP16. The intermediate exponentiated values can easily exceed the maximum representable value of FP16 (65504), resulting in `inf` and `NaN` outputs [3, 22]. `torch.autocast` explicitly keeps `Softmax` in FP32 to prevent this [3].
* **Vision Transformer (ViT) Specifics:** A layer-wise sensitivity analysis of transformer architectures revealed that the **MLP/FFN blocks** are the most sensitive components to quantization, contributing more to overall error than even `LayerNorm` or the attention projections [29]. The linear up- and down-projection layers within these blocks are the primary source of this sensitivity. Furthermore, the residual `Add` connections in transformers can accumulate errors, as small-magnitude outputs from a sub-layer can be "swamped" (lose their precision) when added to a large-magnitude value from the residual path [29].

Quantifying the Degradation

While specific PSNR or SSIM benchmarks for this comparison are not available, the degradation measured by mIoU for segmentation is severe and well-documented.

* **Catastrophic Failure:** One study on an EfficientViT-SAM model reported that converting the encoder to FP16 with `.half()` resulted in a "catastrophic failure," where the model produced no meaningful segmentation mask and the mIoU score dropped to near-zero [29, 30]. Other tests showed imprecise masks and mIoU scores falling to the 0.4-0.5 range [29].
* **Negligible Loss with `autocast`:** In contrast, using `torch.autocast` is widely reported to result in minimal to no accuracy degradation, often less than 1%, while providing significant speedups [20, 27].

Production Deployment Best Practices

The consensus from framework developers and production engineers is to avoid a persistent `model.half()` conversion.

* **PyTorch Inference:** The officially recommended method is to keep the model in FP32 and wrap the inference call in the `with torch.autocast(device_type="cuda", dtype=torch.float16):` context manager [20, 27].
* **TensorRT Deployment:** For deployment with NVIDIA's TensorRT, the correct workflow is to start with a full FP32 model (in ONNX or Torch-TensorRT) and instruct the TensorRT builder to create an optimized FP16 engine [Finding 7]. The builder has its own sophisticated logic to determine which layers can be safely run in FP16, a process that mirrors the `autocast` philosophy. Providing an already-converted `model.half()` graph to the builder is risky because it bypasses these safety checks and may bake in numerical instabilities before TensorRT can optimize them [25].

Recommendations

1. **Prioritize `torch.autocast` for Inference:** For any mixed-precision inference within PyTorch, use the `torch.autocast` context manager. It provides the performance benefits of FP16 while automatically protecting against the quality degradation seen with a blanket `model.half()` conversion.
2. **Avoid Blanket `model.half()` for Deployment:** Do not use `model.half()` to create persistent FP16 weights for segmentation, matting, or transformer-based vision models. The risk of catastrophic quality loss is significant and well-documented.

3. **Deploy FP32 Models to Inference Engines:** When exporting a model to an optimized runtime like TensorRT or ONNX Runtime, always start with the full FP32 model. Enable the FP16 or mixed-precision mode within the runtime's tools, which are designed to perform this conversion safely and optimally.
4. **Use Manual Patching as a Last Resort:** If `model.half()` is unavoidable due to strict memory constraints or legacy systems, manually identify and cast sensitive layers back to FP32. Start with `nn.BatchNorm2d`, `nn.LayerNorm`, and any module using `nn.Softmax`, and validate performance rigorously.
5. **Validate with Task-Specific Metrics:** Regardless of the method used, always validate the final model's quality against an FP32 baseline using appropriate metrics for the task (e.g., mIoU for segmentation; MSE, SAD for matting).

Limitations & Caveats

- The primary limitation of this research is the absence of publicly available benchmarks that directly compare `model.half()` against `torch.autocast` using PSNR, SSIM, or MSE metrics for image segmentation and matting models.

- The quantitative evidence for quality degradation is primarily based on the mIoU metric for semantic segmentation. While the underlying causes of numerical instability are universal, the specific impact on matting quality (measured by MSE/SAD) is not as precisely documented in the available sources.

- Much of the evidence, while consistent, comes from community discussions, blog posts, and GitHub issues rather than peer-reviewed comparative studies, though it aligns with official documentation from PyTorch and NVIDIA.

Follow-up Questions

1. What is the precise PSNR, SSIM, and MSE degradation for a state-of-the-art matting model like ViTMatte when comparing `model.half()` vs. `torch.autocast` on the Composition-1k benchmark?
2. Can Quantization-Aware Training (QAT) effectively recover the quality loss from a full `model.half()` conversion for a model that initially fails, and how does its final quality/performance compare to a post-training `autocast` approach?
3. For a model deployed via TensorRT, what is the final, measured quality and performance difference when the builder starts with a `model.half()` graph versus an FP32 graph with the FP16 flag enabled?
4. How does the sensitivity of MLP/FFN blocks in vision transformers to FP16 conversion change with different activation functions (e.g., GELU vs. ReLU)?
5. Are there specific architectural patterns in U-Net or DeepLab models that make them inherently more or less robust to a full `model.half()` conversion compared to transformer-based architectures?

Sources

[1] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGfSPZ7ryBGNhCTcFlOmmOuc8sNR9AEZ5nr6FNGleAUtcsauVLOXQAsVP4nCDzONLow_J3k7-HBH-E7eluxKd8gUJBCsLuHhO4Y6-3egbIDVQuaQELiZd0IWuJIO9IUKVb_R-9gB68=
[2] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGBizx6Ru0vfvOyrOvLu6jbZw0HsuxXhOkHe9v9es6-Lh9sNMafddzzoTxXt98OIvY6e_9Z3SFmfEu_P4Y2g2USnnLZYJTeiuTaNIB-bdbbBEvHj7h3NMY3rPANQ2s-mXeE6j_u
[3] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHSCIy57DjvF6bPh_AeH4L073-YR1qcO0ENM7WkKOy8bCNoV0KL3nPXXJcPFxRTFPyhML91k18U3S24EKwMeBu6YTDpw1rIVZHuKhnHxNAR9or4RT9EaZhXu1IG
[4] github.com. https://vertexaisearch.cloud.google.com/grounding-api-

redirect/AUZIYQFCRuW1p_34zFnFKPJpcAcLahxo-tSLb9IVQbcnVCTXbgmZFCk9-1wW1S5GNY2AV-
fRD2WeYTPNws29U0B0UE_WF6qckf6hfOEkkzIJVkkhtVwqUIvXf1wf46GWpJzf7v4fBON-z--CRlebZ6kiLQ==
[5] github.com. https://vertexaisearch.cloud.google.com/grounding-api-
redirect/AUZIYQGNEpqzoZDv9MkMYsrroxPvCKAkAbygAbsNWIeEeUMnUdjM5IPxzQYVI747-
AYOwOMOuplKulg4auu62mb2fuHEjvDASRj3t8fVT_DccDT50j_Nx7dKzFJB8-_5EDPCWIK2H5i4iH1DO_Bp
[6] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFK--INfRAv-
qLfUKeSJmMUQK9vUWwx9hn0TW5RrrimkNz1gwAbO-
oPbLIl1Q3LFJbsTyLKBwbAuyqFwZDMbBcQmVVCZRu39NjfWtWDvwo9CtdpAwTv7NfolaQwQmrQB8-
hutFdzduWFc7Kz1SuQKx3
[7] github.com. https://vertexaisearch.cloud.google.com/grounding-api-
redirect/AUZIYQHYjEdYZeGnpJ5nReg8Tos7pHf-
w9AosUJPuySLIaFy4UQc4VeOJdIE1iEzZi5H9fUIaG3OMdvpZfQEMGmt8roxckqrINuACFVq5mBZW0BHs0tcHz-
ceJBhv1FC9tfH0-zfoV6u5LDHhEs=
[8] github.com. https://vertexaisearch.cloud.google.com/grounding-api-
redirect/AUZIYQEiX7liRay9_W6fbfA6Od_CptoOkRVvc-Bh3hEK3LUVj2r1GkX3TR8MUMBgI3cd3O7tv-FZ1EKXteiJon-
CvtTV4-VgKORMSO8pQn2ENFvvRWC2mrwF8p9x_F11sf9IwG8sOOjSdUQ7Bo7J
[9] github.com. https://vertexaisearch.cloud.google.com/grounding-api-
redirect/AUZIYQEqMRRV2qLpktHITrUq21zEYH_9-
SfuxiOmSNq2ZA7se6LjUqZda1UAI4XoPz7A8ThdrwLncinx6ZKIAi8UFtIBMfbT6OxgwDXF4sZpIfX1d5mIC8xWGzevT2Mnqa1HmTGI
M0zA==
[10] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGpmhbhhWJ1ksy1X-
BYXcJQ-ou4Ko_0fYZ0HlyJrUJy7D0sM0Tv-
zPLat9RTjBhsDzwaYUbz1HoQ5ig89woJbJlhgVFnLDJ681ywzaUqAbXTJSexc5N4Q6ODq8xsUnTEM8IBDIlrPWLwp7S
[11] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGtAngNqhZ-
rE6wmZhM_LJ2BKVQ-NWBUP-
Z5CXsqNAf_7LIIhSQ2zUtCNJLBpuekDeSVUwNEz9R_8IDMhfrjLW5afq1p2fzN5SEJVp4VDO5dr_e0jQ3BudQwSw_2tRt0GQDT0qU
[12] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGCVH1G0-
rsgwmdSy7_Z7jHZwKa5JN_nEILDLL-FDUfIy9MrL5LE6_uzv15AxCCZ2O3Q-
lYudz0lIWiBGPsObH7cJQveEr04cBA8IY1Ojov3eJUyccfs1Wq2LozyrPLy63dZuqPhyDGMyU=
[13] reddit.com. https://vertexaisearch.cloud.google.com/grounding-api-
redirect/AUZIYQGKAh08INXEX4hgoC6HAd7kwhhjUhpcaLuMUgD1WIFgKRP7Fjy2G4Jgc3CbkJ3KOzAtGF-M-
MQcMY9XcoPqnpzsHZJQFRAOMhUKfGiK8eE7GkIZFAwnV4xdvQcMpQgubuya7cf1Q5JMgjMULRjLaTDzdD9vKAL2I76VsQRbCec
[14] reddit.com. https://vertexaisearch.cloud.google.com/grounding-api-
redirect/AUZIYQGyK2A9ZnKXLcpqKIEy90bTmrZaQYLBWEyb6Rj1__M69qNBMVvbJ8QBrM2owWMu8Jdb8dXp6BMyDadrLHiVD3
wIDYNuU94O4aIOQaIHTexOrnefqpgb5hd0umytC9jzBExow0732Y9cR7a37T4MbhHjy3ywk7XkaUPrtFKP9C8FoYnQloG6FNFcfrkM
[15] reddit.com. https://vertexaisearch.cloud.google.com/grounding-api-
redirect/AUZIYQF_q6LKh74UrVOXfanVFU21uV8ZSEK0cb22snH4WgsEqKsMGbLcykeoDoEuNK5s63nz8G8u2jKUDPcGjdrExfVarv
SZtOTpesSw5a04pQfNsJGA_SrTuW49QAtJo0
[16] github.io. https://vertexaisearch.cloud.google.com/grounding-api-
redirect/AUZIYQGmBDkb2L3KksfjEujSluOngQJwAhnNEzFHsf3cPW24Xn6Txz-
zE_zQrX_z5X1S0hjWEiSIILeUE_Ihh_nuuVKCmWDTVnxXaZoQfJO9TEEwp5scI5N6xfUcmV_HOHdffUK-hL08o-
XJ6Ij4KutK1PanSOjphNe0A2Iv0DOQbhBTNw8Xemeeuri7y8xtSIFhWSGN
[17] github.io. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFswUhmMkHyljFK-
ngSgARXVSEsmtHGrKkSJlaLc94jsDPXUH3GrW18PcBXBJh4M_elmw-1qxKHa3f39FI6M0Z-82gx-
NTdIjtouv41grJ2j1WAIYRl_pe0E45ewtsaaug_LwjmYlH5JqiAU0GGkvk3CIPi3uuS3CJyojDscPNwiuKt0f788sHaCHu2ksDzLEIzUrxC
zxmjhbTeRt1sdb0Nc8GE=
[18] nih.gov. https://vertexaisearch.cloud.google.com/grounding-api-
redirect/AUZIYQECOW8ReJX4gkLererDBaMgPeN6KIhIIYZ9Y9c4cvkYeYIcoWVg1M1WhrfkO_A9R2N4z_jNaOmhp8xpPTL6JYG5-
5IUxcrF7zFjdZb-CNLgX0tMrgchYT3OK9ghmLjojPHQ4GPWbyaQlnq_Lw==
[19] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-
redirect/AUZIYQHuHsMirLVJsKbLvbtZSB4WjXALHq13T7BKnAt78nAkyQXJVEOE7UOPPSXIZt_IvisPGQ3JZaW1OUb0hLsRgwjz8D9
a4aqCEUUiej_H7Aj666gn-GJP8HHs04JwF9n4N-u38s7ag==
[20] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-
redirect/AUZIYQFz7sDrHYOf8I5S3EnZpW4LvLkII8V9-
dEmFcWWbN8nXc0rjIyqnGYEzg5csczmWeNB1_36N2FY3Je_socGIzV-JUNp1eo3I-
AKOYX1EIjm36Qloc62jRkk9IUWX5yH-sNcng0cD_-n4OzdelyXTUK8z4iE_Euzexz-
_RYLcu_zG0PWXrbU2b6sydzJo3iNjTJnVn8t2Hc0Wjnu4aSmbmk0iJAnEOph4JY=
[21] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGK5eNWFKo_D-

yL7yI3qNLsi17e5r1d_cgXhtMz_6KqoLJmKOWnSyZ3qPTfUKOVs5O2Z3IJYkkXsIV5qNJ43tWF2zqDUkgcTW6aBx8TC21AfsRmVDt
JSZE9Bktwr42y8aH1gQB3F9wNZq8IlbdxgPQGF3h9Ir2xqOt0Fpndc2cjLbjrbHBXRzGpKUg=
[22] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFuCrzHgW7ORyLzxtL6q_ej9p8tk9ZTtVzIc9SVuJp5tJcCnapC0bKIr5wqOorrZ7LVREY63n726ZpVQ1X3D7oTb88Ifr
0gbBAjWMdcqw8vvHVGM6ovi7MwNxdOnoicIP4EZuH-L_0glzfLjXZrk-LqkIH86X1
[23] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQE1CbdIOCaKvtDhYyc5Q61uzIMiNrrkgzMUZQLDx3xY3Mc_tJFezoa-f1rWUSnpiQs7K3EgoJThNkiDKk0D2M-AeYTxjHvXPros9lRnipOZDgr1kXcmN1U6GmryLI5NtkRFBP8JvA5FETF7s-cSd6Z3pRqObXS5u1aWbYuMAp2yoJC3EtX6ANHcfalpKmlAq5zlhMKKVfNHCOcdJcR7RDCRpbP8TqUKodDF3Cw=
[24] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQEvPMJLqI9LjM5FKt3vIx6DtIndQpxrJeIKF9YUBjypTBPADeWB1iHGuHqep7ABLLBU_2MEXpj1xPn3KSwDnLRXTdU8
OX-rIcHii39BUoSbI_zp_M8QNhcyZRzdJTYVzP_nuetsbjyfKMbVsXpCrotxU8MVWhrMkrH-v1A0Lm
[25] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHMWYEEzS5hlLcOt6n5GgEpxnAiu0lhlFimXntVBCbtsx1281vGqSwNe-NTq4LPKCFVbWD0cjQa24GLjI0DD5Y79Z4tmHNFKiAXic2rLiFjjSYeh8ic2BTZbgx5wBgwMb9hWH5KJPB3ZpaH0G9DBR12ZBfVSz
[26] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQEQ2j-Z4JnKyt2rWKI67ZirVmjZK3oOvNHWI4hKZQLe-Agyz-B8nC2amoXs_iwfHzrbdaoINczAtI0eIbIUwOBpRPLd36p85HQBKNcmARWcgVuOSTFr1u7tU682MdATFqlCcM5ZQ-QZLeE0ib0dcHW__BPMURbAKikayyy7NTT6NZm6N-TCJvYdsi3Uh2eqknU2W4jLSwrnafwsK1bKyFUFovHOGtaiii7utkb4YJYD56E9B532Ud9uVKHIK0KkxIqdRIHmPJLLvrLoy3cA
[27] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGM79mpgc4RMBFG89zqDuepfuCpXf1YZiMejo9lhUCadOeSIdmERBvuMh0dkGoh1HMFir4O5im__ck-B5OhTLtno_TY4f-koOLkpcL3LYxwqfK7V3lZ9ayD4kj6bMRuIDyLI_Ev-qWpkXKWUmqFtmSEBWxnodLIbu0s2ZzZLcCZgoXeCNbwOgla-rcY1FrCz7O5qbLDIgoOtH22urrSFRQU7Kdh_BUArvE=
[28] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHiSj9rQzxqmAOHqyw_NM4fHFkWa-Eru0YpPZIm85iTZIwXzFqIwjj4Ugn-Ev5B-MWJ3C1KR9VH-K5gy3v51y02SK3pnLBIdAvmU5P2YI9xfE57Q1oTO-1pcTIrtdxONZ4QczdZ1PSOOIOFo7xXIqYZFz7LcQiuvz7f2d7xCI572DbjM15X7HHCt1QreMcQD6HrW5_KS5R44EJ-USpuOMh1URwnXaXuY_g=
[29] researchgate.net. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQEHr7eQ8HxRXzW7uXo56_2jhK30WjU56mZEEZd-QF3dmaox4wImjdQBfmB5uLcipr13b6JUa4wNY7TjJKdX9Qn9WEVqgFQDKARjOO1sJ6A9tDge6PQDJAX-yQFPgdoAyjyimELLqsPqS-j9ihaVI41dofSSccvR6g7JNSs97lyhh02fOHGu9ypOXmGwgVELKds0tSce58V-SI1crxSt5×9y7WMn84NeDcTFiB7MZuJjOiodVNeMA1iWMMdf7xjGZLwFf3CvmjxKHHLpqOT04oBeFyQ7JidfI1P7cLk4uHqRI5qTRW
oeiiJpIraz6jEqS0yIzqxJt-YGPBa696qlkJHDBQW4vRXFBg9JI6uFBROctFKQk2rJ_Ob27Q7fEM0JI0J50egkJw==
[30] researchgate.net. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGxi9sP4-C1U7tg9XWAJfhPZVaLWIhy40WYXHShO2TcJZPY9PEOUrWfES8padHIcHGadP2WEWk66tiGHMJ_IXTfoABRHNs-k_H5PaqjCGFiY25IIw_EOwTizCvkYXh26azIumQWjMO9rBhgh2hDUcnItCCGe2WEp6jtrx0t4WCI-3c2ckK5HkyQeZ8YfGm7PEeIIyhCvtSN-GgV1_Pc3M39SUOHBTP7fDrz1jzBQG8kL4LbpFXtOviW0EbvKZI8pEp6h1I=

Methodology

- **Backend**: VERTEX API
- **Model**: gemini-2.5-pro
- **Research Depth**: 3 (follow-up iterations)
- **Research Breadth**: 3 (parallel queries per iteration)
- **Total Sources Evaluated**: 65

- **Quality Threshold**: 40%+ (verified sources only)
- **Duration**: 320.6s
- **Synthesis Method**: AI-assisted cross-referencing and verification

---

API Costs

Metric	Value
Input Tokens	25,817
Output Tokens	8,039
Total Tokens	**33,856**
Input Cost	$0.0323
Output Cost	$0.0804
Total Cost	**$0.1127**

Report generated by ez-deep-research MCP